MINI PROJECT REPORT

on

DISTRIBUTED COMMUNICATION NETWORK

Submitted by

ROOPESH O R (20323085)
SIDHARTH T P (20323093)
SUMAYYA PUNNOTH (20323100)
WASEEM ANWAR (20323108)

in partial fulfillment of requirement for the award of the degree of

BACHELOR OF TECHNOLOGY
in
ELECTRONICS AND COMMUNICATION

DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI - 682022

APRIL 2026

DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI-682022

CERTIFICATE

Certified that the mini project report entitled ***"DISTRIBUTED COMMUNICATION NETWORK"*** *is a bonafide work of* ***ROOPESH O R, SUMAYYA PUNNOTH, SIDHARTH T P, WASEEM ANWAR*** *towards the partial fulfillment for the award of the degree of B.Tech in Electronics and Communication of Cochin University of Science and Technology, Kochi-682022.*

Mini Project Guide **Head of the Division**

Acknowledgment

The satisfaction that accompanies the successful completion of the project would be incomplete without the mention of the people who made it possible. We consider ourself privileged to express gratitude and respect towards all those who guided us through the completion of this project. We express our indebtedness in God Almighty for the successful completion of our project. First and foremost, we express our sincerest regards to our project guides, Unni A M, Mumthaz and Krishnapriya C V for their valuable inputs, guidance cooperation and constructive criticism throughout the duration of our project. We deeply express our sincere thanks to our Head of Department, Dr. Deepa Sankar for encouraging and allowing us to present the project at our department premises. We take this opportunity to thank all our lecturers who directly or indirectly helped our project. We pay respect and love to our parents and all other family members and friends for their love and encouragement throughout our career. Last but not the least we express our thanks to our friends for their cooperation and support

Abstract

This project presents the design and implementation of a distributed, low-cost communication system specifically for person localization in a forest using a mesh network. The system is built on STM32 and uses nRF24L01+ transceivers to achieve 2.4GHz wireless links with across the network. Unlike traditional star topologies, the nodes operate within a software-defined mesh, utilizing a controlled flooding algorithm to ensure near fault tolerant message delivery across nodes, even in challenging RF environments. A primary feature of the implementation is the custom path-recording within packets transmitted by node; each data packet carries a telemetry header that logs the unique identifiers of every intermediary relay node. This mechanism allows for tracking of person's location within the network's topology and facilitates fast rescue operation.

Table of contents

1. Introduction **1**
1.1. System Architecture 1
1.2. Protocol Design 2
2. Background Study **4**
3. Component Selection **6**
3.1. STM32F103 6
3.2. nRF24L01+ 7
3.3. Project cost 8
4. System Overview **9**
4.1. Hardware 9
4.2. Software 10
4.2.1. Transmitter 11
4.2.2. Receiver 12
5. Results and Discussion **13**
5.1. Latency Measure 14
5.2. limitations 14
6. Future Scope **16**
7. Bibliography **17**
Appendices **18**
A.1 Transmitter Program 18
A.2 Receiver Program 21

List of Figures

Figure 1.1 Intended organization of nodes 2
Figure 2.1 Star and Common Bus topologies 4
Figure 2.2 Mesh network topology 5
Figure 3.1 STM32F103 development board 6
Figure 3.2 nRF24L01+ and its pinout 7
Figure 4.1 Circuit diagram 9
Figure 4.2 Transmitter (left) and receiver(right) modules 10
Figure 4.3 Settings to apply while flashing the program to STM board 10
Figure 5.1 Serial output from Transmitter, Receiver and Repeater nodes (left to right) 13
Figure 5.2 Round trip Transmission delay in direct communication 14

List of Tables

Table 3.1 Cost estimate 8
Table 5.1 Latency for direct communication 14
Table 5.2 Latency for 1-hop communication 14

List of Abbreviations

ACK: Acknowledgment
AODV: Ad-hoc On-demand Distance Vector
COTS: Commercial off-the-shelf
ESB: Enhanced ShockBurst
GNSS: Global Navigation Satellite Systems
GPIO: General Purpose I/O
I2C: Inter-Integrated Circuit
IDE: Integrated Development Environment
IoT: Internet of Things
LOS: Line-of-Sight
M2M: machine-to-machine
MANET: Mobile Adhoc Network
SPI: Serial Peripheral Interface
SRAM: Static RAM
UAV: Unmanned Aerial Vehicle
USART: Universal Synchronous/Asynchronous Receiver Transmitter
VANET: Vehicular Adhoc Network
WSN: Wireless Sensor Netwok

Chapter 1: Introduction

Communication infrastructure is the most critical logistical component of any Search and Rescue operation. In deep, dense forests, traditional communication networks such as cellular grids or centralized VHF radio repeaters are either non-existent or rendered ineffective due to severe signal attenuation caused by foliage and uneven terrain. When search teams or lost individuals operate in these disconnected zones, the lack of a reliable communication lifeline drastically reduces the probability of a successful rescue.

To overcome the limitations of centralized networks, decentralized Wireless Sensor Netwok (WSN) offer an alternative. By deploying a localized, ad-hoc mesh network, communication can be maintained by "hopping" signals from one node to another until they reach a central command station. This project demonstrates the design and implementation of such a system: a distributed multi-hop communication network designed specifically to relay critical state changes (such as an SOS signal) across separated nodes.

1.1 System Architecture

The core objective of this prototype is to validate the feasibility of decentralized packet routing using low-cost hardware. The physical demonstration consists of a localized four-node network:

- One ***Originating Transmitter***: Simulating an edge device triggered by a user in distress.
- One ***Destination Receiver***: Acting as the gateway or rescue command center.
- Two ***Relay Nodes***: Acting as repeaters. Because these nodes are battery-powered, forest guards could place these modules in different parts of forest to instantly establish a network.

The computation of each node is handled by an STM32 series microcontroller, which handles interfacing of nRF24L01 RF module. These four nodes are expected to be organized in a manner shown in Figure 1.1.

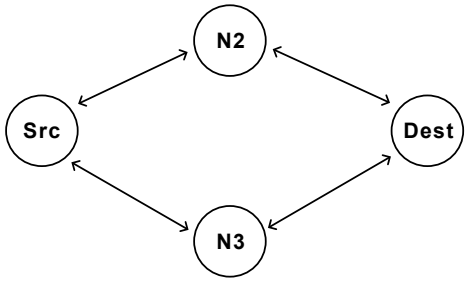


Figure 1.1: Intended organization of nodes

1.2 Protocol Design

A primary decision in this project was the intentional bypass of heavy, conventional mesh routing protocols like Ad-hoc On-demand Distance Vector (AODV) or Zigbee. While these protocols are highly efficient in static, enterprise-grade networks, they require significant effort to get started and to maintain routing tables and calculate shortest-path vectors. Since no GNSS system is incorporated the node distance is not directly computable.

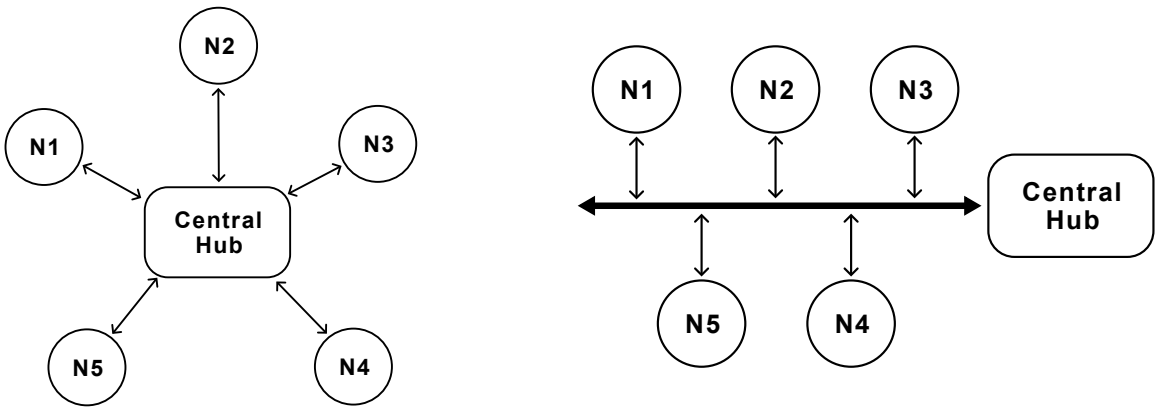
Considering the data available to each node we decided to implemented Controlled Flooding. In a flooding architecture, an intermediate node simply broadcasts any packet it receives that is not addressed to itself. To prevent the network from collapsing under infinite routing loops (known as a “broadcast storm”), the flooding is strictly controlled via software. Each node maintains a localized “seen stack” – an array of details of recently transmitted packets. If a packet’s signature exists in the stack, it is dropped; if it is new, it is forwarded. This ensures redundancy and that an SOS signal will aggressively find the fastest physical path through the forest to the receiver, without the nodes needing to map the network topology in advance.

In emergency communications, “fire-and-forget” transmission is unacceptable; the originating node must know if the distress signal successfully reached the command station. To signal the successful data delivery, the system implements an application-layer Acknowledgment (ACK) mechanism. When the final destination node successfully receives the targeted packet, it generates a distinct ACK packet and routes it back through the mesh to the original transmitter. The originating node after transmitting SOS goes to listening state, and upon receiving this targeted ACK, the node provides

physical feedback (via an LED indicator) to confirm that the transmission successfully reached the command station.

Chapter 2:
Background Study

Historically, network systems have been studied for wide area communication [1] and distributed computing. In earlier days sensor networks relied on centralized, ***star topology*** architectures. In these networks, all edge devices communicate directly with a central gateway. While simple to implement, star topologies possess a single point of failure and are strictly limited by the physical transmission range of the central node . In complex environments – such as industrial facilities with heavy RF attenuation, dense agricultural fields, or robotic swarm deployments – maintaining direct line-of-sight to a central hub is often physically impossible or expensive. A ***common bus topology*** was also considered but it suffers from the same problem – need of central orchestrator.



(a) Star Topology (b) Common Bus Topology

Figure 2.1: Star and Common Bus topologies

In these scenario decentralized ***Mesh Networking*** has emerged as a viable solution [2]. In a mesh topology, individual nodes can act as originators of data and/or active relays (routers) for their peers. This cooperative data forwarding extends the effective range of the network far beyond the capability of single transmitter system and provides much higher fault tolerance; if one node fails, the network can re-routes traffic through other existing paths. These network are self-organizing and does not require a central orchestrator.

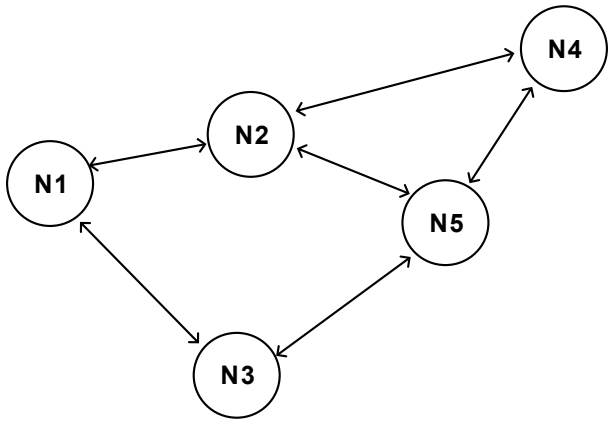


Figure 2.2: Mesh network topology

Despite the advantages of mesh topologies, implementing them in highly constrained environments is a significant engineering challenge. Standard mesh protocols, such as Zigbee has performance and interoperability issues for large networks. Furthermore, many commercial mesh solutions are proprietary or rely on heavy, power-hungry transceivers.

There are more complex systems like Mobile Adhoc Networks (MANETs), and Vehicular Adhoc Networks (VANETs) which are suited for very dynamic environments. We also explored *Meshtastic* a sophisticated off-grid mesh networking LoRa protocol [3]. However, the problem our project tries to tackle involves a relatively static environment where nodes remain in their place for long run. Hence routing scheme such as **Controlled Flooding** [4] best suited for our problem domain. Controlled flooding is easier to implement while being light on resources.

Chapter 3: Component Selection

The key components used are STM32F103 (C6T6 variant), and nRF24L01+. Other minor components used are listed in Section 3.3: Project cost*: Project cost*

3.1 STM32F103

The STM32F103 is the main processing unit of each node in the network. It is based on the ARM Cortex-M3 architecture and is responsible for controlling all node operations, including packet creation, packet forwarding, routing logic, path recording, and communication with peripheral devices. The microcontroller offers sufficient processing speed, low power consumption, and multiple communication interfaces such as SPI and UART, which make it well suited for embedded wireless applications. In this project, the STM32F103 handles the software-defined mesh logic and manages the interaction with the nRF24L01+ transceiver. The microcontroller has following key specifications:

- 16 KiB flash memory and 6 KiB SRAM.
- 72 MHz Clock frequency
- Supports 2.0 to 3.6V Input
- Has one SPI, one I2C, 2 Universal Synchronous/Asynchronous Receiver Transmitter (USART)
- 37 General Purpose I/O (GPIO)

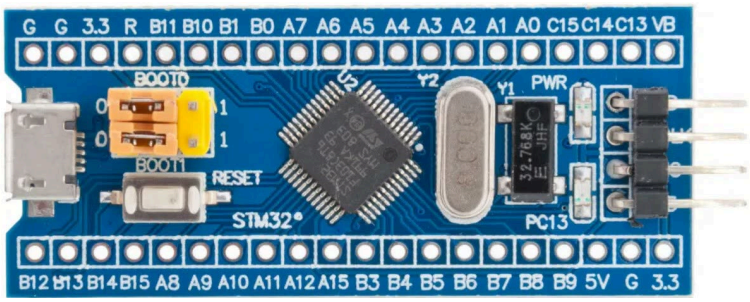


Figure 3.1: STM32F103 development board

There are many STM32 variants (such as F401) in market however we choose this particular model due to the requirements:

- Very low cost
- No need of floating point operations (hence not considering F401)
- Enough number of GPIO pins for future expansion of peripherals.

However programming the board was not a easy task. ST Microelectronics recommends using ***STM32CubeIDE*** – the official IDE for STM boards – for creating and flashing programs onto the development boards. The board we are currently using is unfortunately unsupported by the IDE. Hence the board was programmed using Arduino IDE by installing STM board package [5] and setting appropriate settings (shown in Figure 4.3 in *Section 4.2: Software*) while flashing. The programming was hence written in Arduino's C++ language.

3.2 nRF24L01+

The nRF24L01+ module is used to establish wireless communication between nodes in the mesh network. It operates in the 2.4 GHz ISM band and supports low-power, short-range digital communication with good data transfer rates. The transceiver communicates with the STM32F103 through the SPI interface and is responsible for transmitting and receiving packets between neighboring nodes. In this project, the nRF24L01+ enables multi-hop communication through controlled flooding, allowing packets to propagate across the network while recording the relay path followed. The module has following specifications: [6]

- Very low standby current of 22μA
- Supports input voltage of 1.9 to 3.6V, (which can be powered by STM board itself).
- 12 mA current consumption during operation.
- Data rate of 250Kbps
- Data transfer through Serial Peripheral Interface (SPI)

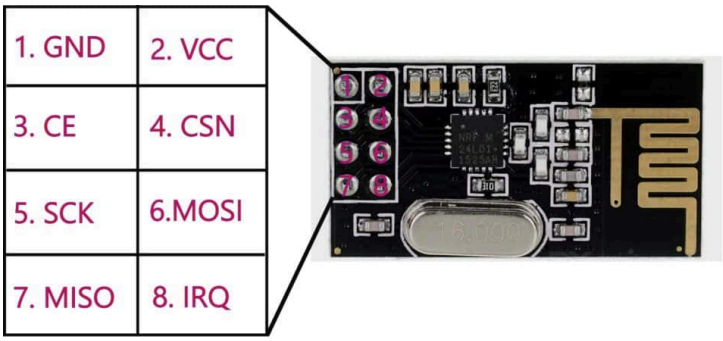


Figure 3.2: nRF24L01+ and its pinout

We considered other alternatives such as nRF24L01+PA+LNA version which has power amplifier for transmission and a low noise amplifier for receiver extending the Line-of-Sight (LOS) communication range to 1km. However the module was five times expensive than the base model significantly rising the budget. Moreover due to its extensive range, it is not suitable for the demonstration of our network as we will have to setup large area for testing. Similarly LoRa modules were also not considered due to its wide range and high abstraction from hardware. FS1000A was another module taken for consideration for being inexpensive. But from our past experiences, the module is often unstable and has higher rate of failure. Moreover these modules have poor manufacturing quality.

The nRF24L01 was chosen considering the drawbacks of other alternatives.

3.3 Project cost

Although a breadboard was used for a single node project, it was solely ment for testing purpose, the total cost listed covers prototype board can in place of breadboard.

Item	**Unit Cost**	**Total**
STM32	90	450
nRF	60	300
LED	2	8
Resistors	1	4
Prototype board	60	60
Push button	5	5
Total		**827**

Table 3.1: Cost estimate

Chapter 4:
System Overview

The system is made with portability in mind. The development board and nRF module are inserted onto header pins soldered on a small section of prototype board. The algorithm is custom developed based on already available Controlled flooding algorithms [4], [7]

4.1 Hardware

The nRF module is interfaced to STM board via SPI through hardware SPI peripheral of STM32. An indicator LED used to indicate the status of transmission is attached to `PB11` pin of STM. Built-in LED in the development board – which is accessed by `PC13` is used to indicate reception of radio packet reducing the need for additional LED and current limiting resistor.

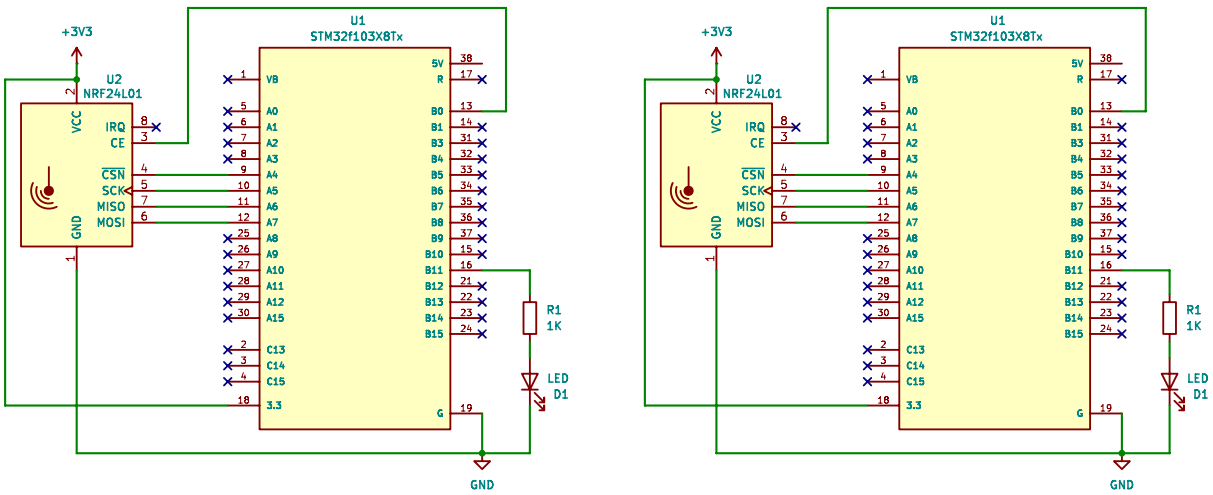


Figure 4.1: Circuit diagram

While implementing, we soldered female berg strip onto prototype board. For transmitter, we were modifying the quite a while, so it wasn't transferred to prototype board.

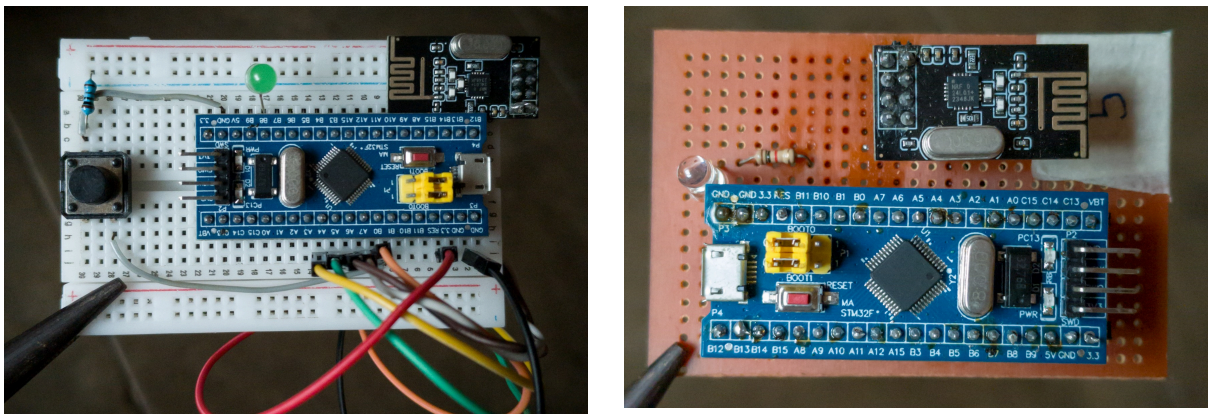


Figure 4.2: Transmitter (left) and receiver(right) modules

4.2 Software

This section outlines the algorithm of custom controlled flooding method. [8]

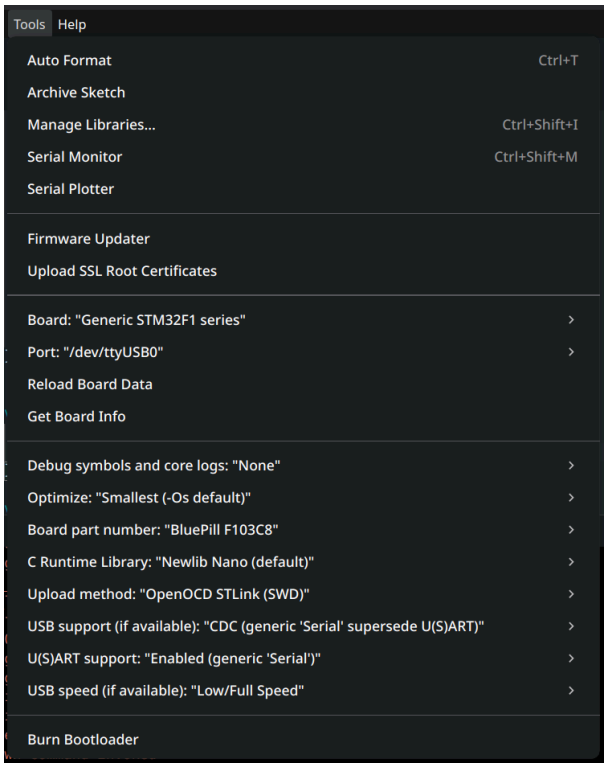


Figure 4.3: Settings to apply while flashing the program to STM board

4.2.1 Transmitter

Algorithm 4.1: Transmitter Initialization

```
Init Button and LED GPIO pins
LED_SEND ← HIGH
if Radio not responding then
    (Infinite loop)
end
Set radio in listening mode
```

Algorithm 4.2: SOS Event and Transmission Start

```
if SOS BTN pressed and not waiting For ACK then
    LED_SEND ← LOW
    Rebuild pkt memory buffer
    Transmit pkt
    waitingForACK ← 1
    lastSOS_Sent ← currentTime()
    msgID ← msgID + 1
    Delay 500ms
end
```

Algorithm 4.3: ACK Reception

```
if waitingForACK and payload received then
    pkt ← Received payload
    if pkt.msg_type != SOS or pkt already seen then
        BREAK
    end
    if pkt destination = This Node then
        Blink Indications
        Save packet signature
        BREAK
    end
    if pkt expired then
        Save packet signature
        BREAK
    end
end
```

4.2.2 Receiver

Algorithm 4.4: Receiver Initialization

```
Init LED GPIO pins
LED_SEND ← HIGH
if Radio not responding then
    (Infinite loop)
end
Set radio in listening mode
```

Algorithm 4.5: Handle arrival of new package

```
if Payload Received then
    pkt ← Receive Packet
    if pkt already seen then
        BREAK
    end
    if pkt expired then
        Save packet signature
        BREAK
    end
    Append This node's ID to pkt.path
    if pkt destination = This Node then
        Blink RX LED
        Save packet signature
        Construct ACK packet using pkt buffer with dest = original sender
    end
    Save pkt's packet signature
    Transmit pkt
    Blink TX LED
end
```

Chapter 5:
Results and Discussion

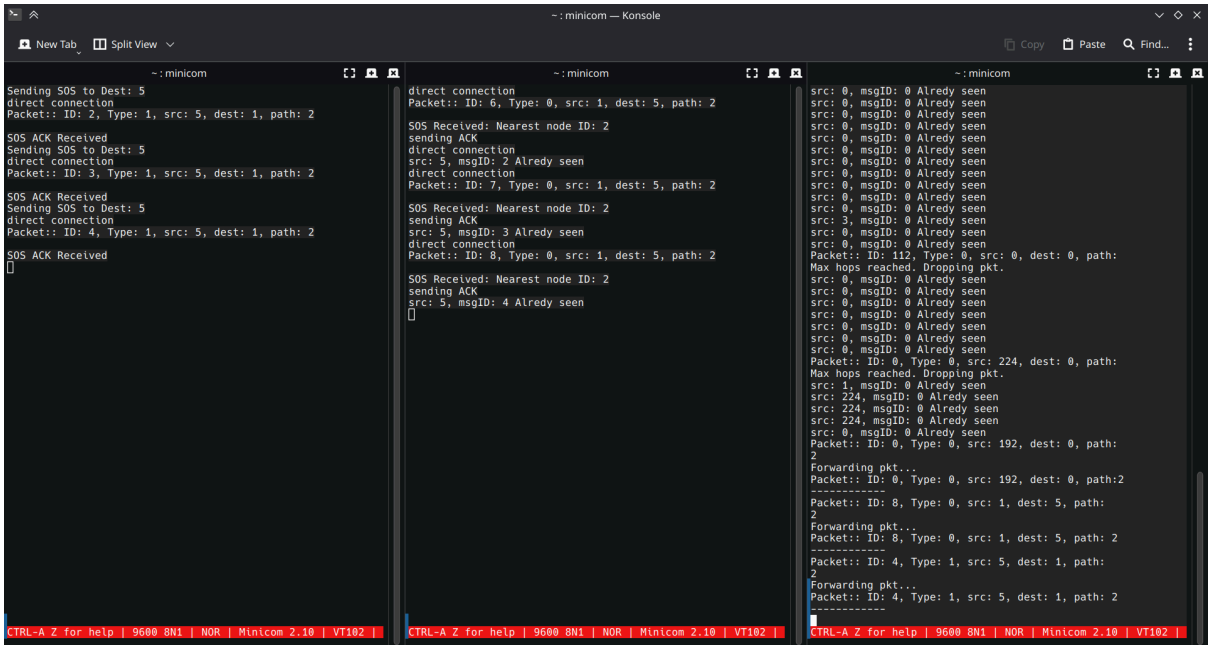


Figure 5.1: Serial output from Transmitter, Receiver and Repeater nodes (left to right)

5.1 Latency Measure

```
SOS ACK Received
1301
Sending SOS to Dest: 5

SOS ACK Received
1306
Sending SOS to Dest: 5

SOS ACK Received
1312
Sending SOS to Dest: 5

SOS ACK Received
1304
Sending SOS to Dest: 5
```

Figure 5.2: Round trip Transmission delay in direct communication

Latency (μs)	1301	1306	1312	1304	1304	1304	1305	1304	1304
Average	**1304μs**								

Table 5.1: Latency for direct communication

Latency(μs)	32198	32210	32023	32034
Average	**32116 μs**			

Table 5.2: Latency for 1-hop communication

5.2 limitations

The physical RF layer of this prototype is powered by the standard nRF24L01+ transceiver (without the external Power Amplifier and Low Noise Amplifier stages). While this choice keeps the prototype highly cost-effective and energy-efficient, it introduces specific physical and programmatic limitations:

- Range Constraints: Without a PA+LNA, the transmission range of the standard nRF24L01+ is strictly limited (typically 10 to 30 meters depending on obstacles). In a real forest deployment, RF absorption by trees would further reduce this range, necessitating a higher density of relay nodes.
- MAC-Layer Abstraction: The nRF24L01+ chip features a built-in hardware protocol called Enhanced ShockBurst (ESB). ESB automatically handles hardware-level ACKs and auto-retransmissions (e.g., automatically resending a failed packet up to 3 times, or up to 15 times depending on register settings). While convenient for simple point-to-point links, this hardware abstraction removes granular software control from the developer. It makes it difficult to implement precise, custom collision-avoidance algorithms at the MAC (Media Access Control) layer, as the hardware dictates the timing of the retransmissions independent of the STM32′s software mesh logic.

Chapter 6:
Future Scope

Just fill this thing.....

- sending high volume of data (increasing bandwidth)
- Ad-hoc possibility
- vehicle ad-hoc
- sensor networks (wild fire detection..)
- etc...

Chapter 7: Bibliography

[1] Lawrence G. Roberts and Barry D. Wessler, "Computer network development to achieve resource sharing."

[2] Zhanserik Nurlan and et al., "Wireless Sensor Network as a Mesh: Vision and Challenges."

[3] Stephen Cass, "Build Long-Range IoT Applications Fast With Meshtastic." [Online]. Available: https://spectrum.ieee.org/build-iot-apps-with-meshtastic

[4] A. Rahman, W. Olesinski, and P. Gburzynski, "Controlled Flooding in Wireless Ad-hoc Networks." [Online]. Available: https://web.archive.org/web/20170210205207/http://www.senserf.com/pg/PAPERS/tarp1.pdf

[5] B.Montanari, "How to program and debug the STM32 using the Arduino IDE." [Online]. Available: https://community.st.com/t5/stm32-mcus/how-to-program-and-debug-the-stm32-using-the-arduino-ide/ta-p/608514

[6] Nordic Semiconductor, "nRF24L01 Single Chip 2.4GHz Transceiver." [Online]. Available: https://cdn.sparkfun.com/datasheets/Wireless/Nordic/nRF24L01_Product_Specification_v2_0.pdf

[7] Fatemeh Nourazar and Masoud Sabaei, "An efficient flooding algorithm for Mobile Ad-hoc networks."

[8] M. Scott Corson and Anthony Ephremides, "A distributed routing algorithm for mobile wireless networks."

Appendices

A.1 Transmitter Program

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define MAX_PATH_SIZE 27
// node ID is never 0
#define DEST_NODE_ID 5
#define THIS_NODE_ID 1
#define MAX_PACKET_SIZE 32
#define MAX_SEEN_STACK 20
#define BTN_SOS_PIN PB1
#define LED_SOS_SEND PC13
#define LED_SOS_ACK PB8
typedef struct __attribute__((packed)) {
  byte msgID;
  byte msg_type; // 0 - sos, 1 - ack
  byte ttl;
  byte src;
  byte dest;
s } Packet;

// Radio Pins: CE = PB0, CSN = PA4
RF24 radio(PB0, PA4);
const uint64_t address = 0xF0F0F0F0E1LL;

byte seenTxrStack[MAX_SEEN_STACK] = { 0 };
byte seenMsgIDStack[MAX_SEEN_STACK] = { 0 };
int seenStackTop = -1;

byte msgID = 1;
Packet pkt;
byte waitingForACK = 0;
long lastSOS_Sent;
void setup() {
  Serial.begin(9600);

  pinMode(BTN_SOS_PIN, INPUT_PULLUP);
  pinMode(LED_SOS_SEND, OUTPUT);
  pinMode(LED_SOS_ACK, OUTPUT);
  digitalWrite(LED_SOS_SEND, HIGH);
```

```
  if (!radio.begin()) {
    Serial.println("Radio hardware not responding!");
    while (1)
      ;
  }

  radio.openWritingPipe(address);
  radio.openReadingPipe(0, address);
  radio.setPALevel(RF24_PA_MIN);
  radio.startListening();

  Serial.println("Transmitter ready");
}

void loop() {

  if (digitalRead(BTN_SOS_PIN) == LOW && waitingForACK == 0) {
    digitalWrite(LED_SOS_SEND, LOW);
    // Build pkt
    memset(&pkt, 0, sizeof(Packet));
    pkt.msgID = msgID;
    pkt.msg_type = 0;
    pkt.ttl = 10;             // TTL
    pkt.src = THIS_NODE_ID;   // Src node ID
    pkt.dest = DEST_NODE_ID;  // destination node ID

    Serial.print("Sending SOS to Dest: ");
    Serial.println(DEST_NODE_ID);
    transmit(&pkt, sizeof(Packet));
    waitingForACK = 1;
    lastSOS_Sent = millis();
    msgID++;
    delay(500);
  } else if (waitingForACK == 1 && radio.available()) {
    memset(&pkt, 0, sizeof(Packet));
    radio.read(&pkt, sizeof(Packet));
    if (pkt.msg_type != 1) return;

    if (pkt.path[0] == 0 && pkt.dest == THIS_NODE_ID) {
      Serial.println("direct connection");
      // return;
    }
    for (int j = 0; j < MAX_SEEN_STACK; j++) {
      if (seenTxrStack[j] == pkt.src && seenMsgIDStack[j] == pkt.msgID) {
        // ignore
        Serial.print("src: ");
        Serial.print(pkt.src);
        Serial.print(", msgID: ");
        Serial.print(pkt.msgID);
        Serial.println(" Alredy seen");
```

```
        return;
      }
    }

    int i = pathStackTop(pkt);  // no of nodes in path.

    // Destination reached
    if (pkt.dest == THIS_NODE_ID) {
      Serial.println("\nSOS ACK Received");
      digitalWrite(LED_SOS_ACK, HIGH);
      waitingForACK = 0;
      digitalWrite(LED_SOS_SEND, HIGH);
      delay(1000);
      digitalWrite(LED_SOS_ACK, LOW);

      markAsSeen(pkt);
      return ;
    }

    if (pkt.ttl <= 0) {
      markAsSeen(pkt);
      return;
    }
  } else if (waitingForACK == 1 && millis() - lastSOS_Sent > 3000) {
    waitingForACK = 0;
    digitalWrite(LED_SOS_SEND, HIGH);
  }
}

void transmit(const void* data, uint8_t len) {
  radio.stopListening();
  radio.write(data, len);
  radio.startListening();
}

int pathStackTop(Packet pkt) {
  char printBuff[50];
  snprintf(printBuff, sizeof(printBuff),
           "Packet:: ID: %d, Type: %d, src: %d, dest: %d, path: ",
           pkt.msgID, pkt.msg_type, pkt.src, pkt.dest);
  Serial.print(printBuff);
  int i = -1;
  while (i < MAX_PATH_SIZE) {
    i++;
    if (pkt.path[i] == 0) break;
    Serial.print(pkt.path[i]);
  }
  Serial.println();
  return i;
}
```

```
void markAsSeen(Packet pkt) {
  seenStackTop = (seenStackTop + 1) % MAX_SEEN_STACK;
  seenTxrStack[seenStackTop] = pkt.src;
  seenMsgIDStack[seenStackTop] = pkt.msgID;
}
```

A.2 Receiver Program

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define MAX_PATH_SIZE 27
#define DEBUG true
#define THIS_NODE_ID 5
#define MAX_PACKET_SIZE 32
#define MAX_SEEN_STACK 20
#define LED_SEND PC13
#define LED_RX PB11

typedef struct __attribute__((packed)) {
  byte msgID;
  byte msg_type;
  byte ttl;
  byte src;
  byte dest;
  byte path[MAX_PATH_SIZE];
} Packet;

// Radio Pins: CE = PB0, CSN = PA4
RF24 radio(PB0, PA4);
const uint64_t address = 0xF0F0F0F0E1LL;

Packet pkt;

byte seenTxrStack[MAX_SEEN_STACK] = { 0 };
byte seenMsgIDStack[MAX_SEEN_STACK] = { 0 };
int seenStackTop = -1;
byte msgID = 1;
void setup() {
  Serial.begin(9600);
  pinMode(LED_RX, OUTPUT);
  pinMode(LED_SEND, OUTPUT);
  if (!radio.begin()) {
    while (1) {
      Serial.println("Radio hardware not responding!");
      delay(100);
    }
  }
```

```
  radio.openReadingPipe(0, address);
  radio.openWritingPipe(address);
  radio.setPALevel(RF24_PA_MIN);
  radio.startListening();

  Serial.print("Node started. ID = ");
  Serial.println(THIS_NODE_ID);
}

void loop() {
  if (radio.available()) {
    memset(&pkt, 0, sizeof(Packet));
    radio.read(&pkt, sizeof(Packet));
    for (int j = 0; j < MAX_SEEN_STACK; j++) {
      if (pkt.path[j] == THIS_NODE_ID) {
        // ignore
        Serial.print("src: ");
        Serial.print(pkt.src);
        Serial.print(", msgID: ");
        Serial.print(pkt.msgID);
        Serial.println(" Alredy seen");
        return;
      }
    }

    int i = pathStackTop(pkt);  // no of nodes in path.

    // Destination reached
    if (pkt.dest == THIS_NODE_ID) {
      Serial.print("\nSOS Received: Nearest node ID: ");
      Serial.println(pkt.path[0]);
      digitalWrite(LED_RX, HIGH);
      delay(1000);
      digitalWrite(LED_RX, LOW);
      markAsSeen(pkt);
      Serial.println("sending ACK");
      Packet pk2;
      memset(&pk2, 0, sizeof(Packet));
      pk2.msgID = msgID;
      msgID++;
      pk2.msg_type = 1;  // ACK
      pk2.ttl = 10;
      pk2.src = THIS_NODE_ID;
      pk2.dest = pkt.src;
      markAsSeen(pk2);
      transmit(&pk2, sizeof(Packet));
      return;
    }
```

```
    if (pkt.ttl <= 0) {
      markAsSeen(pkt);
      Serial.println("Max hops reached. Dropping pkt.");
      return;
    }

    // Append node ID
    pkt.path[i] = THIS_NODE_ID;
    pkt.ttl = pkt.ttl - 1;
    Serial.println(pkt.path[0]);
    Serial.println("Forwarding pkt...");

    pathStackTop(pkt);

    transmit(&pkt, sizeof(Packet));
    Serial.println("------------");
  }
}

int pathStackTop(Packet pkt) {
  char printBuff[50];
  snprintf(printBuff, sizeof(printBuff),
           "Packet:: ID: %d, Type: %d, src: %d, dest: %d, path: ",
           pkt.msgID, pkt.msg_type, pkt.src, pkt.dest);
  Serial.print(printBuff);
  int i = -1;
  while (i < MAX_PATH_SIZE) {
    i++;
    if (pkt.path[i] == 0) break;
    Serial.print(pkt.path[i]);
  }
  Serial.println();
  return i;
}

void markAsSeen(Packet pkt) {
  seenStackTop = (seenStackTop + 1) % MAX_SEEN_STACK;
  seenTxrStack[seenStackTop] = pkt.src;
  seenMsgIDStack[seenStackTop] = pkt.msgID;
}

void transmit(const void* data, uint8_t len) {
  radio.stopListening();
  digitalWrite(LED_SEND, LOW);
  radio.write(data, len);
  radio.startListening();
  delay(200);
  digitalWrite(LED_SEND, HIGH);
}
```